# RSCL — Algorithm 13: Grounding Validator for Generated Chat Text and Generated-Only Repetition Penalty

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)

Source files described (content hash and state at document build time):

| | | | |
|---|---|---|---|
| hu-telepresence-gazebo/spaceops_v8/chat_grounding.py | sha256 a916f9395247 | 2026-11-11 07:58 UTC | clean @ 10ee643 |
| hu-telepresence-gazebo/spaceops_v8/chat_llm_service.py | sha256 9c6cdcef5c67 | 2026-11-11 08:05 UTC | clean @ 10ee643 |

## 1. Purpose and context

A small local instruct model rewords SwarmChat replies in each aircraft's persona (Algorithm 12). Small models invent numbers, aircraft, events and authority ("I'll take off now"). The validator is the gate between the model and the operator: a generated sentence is shown only when `validate()` returns no violations; otherwise the deterministic text is shown. The design principle is asymmetric: a false rejection costs style, a false acceptance could put an invented fact in front of the operator. The validator is pure standard-library Python, so the observer process can re-check text independently of the model process.

## 2. Inputs and outputs

`validate(text, speaker, evidence, allowed_facts, draft, history, user_text, persona, require_draft_facts=True)` → list of violation strings `"code: detail"` (empty = accepted). **Support sets:** numbers and strings found by recursively walking *evidence* (depth ≤ 8), numbers in *allowed_facts* and the *draft*. User text and history support names but **never numbers or shapes**.

## 3. Algorithm

**Algorithm 13.1 VALIDATE (rule order as implemented)**

Source: spaceops_v8/chat_grounding.py: validate, number_supported, _negated, airborne_state

```
function VALIDATE(t, speaker, ev, facts, draft, hist, user, persona)
  if t blank then return [empty]
  N ← numbers(ev) ∪ numerals in strings(ev), facts, draft;   Sup ← text(strings(ev), facts, draft)
  // format
  if |t| > max(400, |draft| + 80) then too_long
  if sentences(t) > max(3, sentences(draft)) then too_many_sentences       // decimals not split
  if list markers or ≥ 2 newlines then list_format
  stage directions ((laughs), [beeps], *...*, haha/lol), speaker tags ("User:", "Ra:") unless verbatim in draft
  // prompt echo
  echo patterns (FACTS:, rules, instructions, prompt, "you are a ... model", allowed facts, draft:, ...)
  "language model"/"llm" unless the user asked an honesty question
  // numbers
  for each numeral g in t (excluding "3D"/"2D") do
    if not SUPPORTED(g, N) then number_not_in_evidence
  // names
  fleet name not in speaker ∪ Sup ∪ user ∪ history → unknown_drone;  other Egyptian deity names → unknown_drone
  "drone 3", "unit #2" → unknown_drone;  first unexplained capitalised word (non sentence-initial use) → unknown_name
  meta preambles ("here's the response", ends with ":") → meta_text
```

```
  if draft and require_draft_facts then every unit-bearing draft number and every other fleet name in the draft
      must reappear in t (rounding-consistent) else missing_draft_fact
  // authority and persona limits (each skipped if negated within ~6 words of the same clause, or verbatim in draft)
  first-person future/ability/progressive/perfect + flight verb, or authority words (approved, dispatching, wilco ...) → action_claim
  invented safety clearance ("safe to stand under", "cleared as safe") → safety_clearance_claim
  "I'm happy/proud/alive ..." (except glad/happy/pleased/eager to help|assist|chat|...), "I feel/love", "my feelings" → feelings_claim
  outside-world terms (weather, birds, city, music, ...) unless in Sup ∪ persona, or asked by the user and stated as unverifiable → unsupported_world
  incidents (collision, crash, near miss, emergency, damage, ...) not in Sup → unsupported_event
  self-identity (AI, trained, algorithm, neural network, created by ...) unless honesty question → identity_claim
  shape word in formation context whose synonym group is not in Sup → unsupported_formation
  // contradictions with top-level evidence booleans
  fault = true and "no fault/all systems nominal" → contradicts_fault;  fault = false and active-fault claim → contradicts_fault
  airborne(ev) = false and "airborne/flying/hovering" → contradicts_airborne; = true and "landed/parked/grounded" → contradicts_airborne
  telemetry = false and "telemetry is fresh/live/good" → contradicts_telemetry
  return deduplicated violations
function SUPPORTED(g, N)                                  // g as written, d = its decimal places
  return ∃ a ∈ N: |a| = |g|  or  (d ≥ 1 and round_half_up(|a|, d) = |g|)  or  (d = 0 and |a| ≥ 10 and round(|a|) = |g|)
```

**Algorithm 13.2 Generation with generated-only repetition penalty**

Source: spaceops_v8/chat_llm_service.py: build_messages, ChatLLM._generate, ChatLLM._rep_penalty, clean_generation, ChatLLM._handle, ChatLLM.submit

```
function GENERATE(req)
  msgs ← system(persona, strict rules, style hint, FACTS from evidence (≤ 40 lines), optional draft "say the same ...")
         + two fixed example turns (refuse a backflip; "can't verify" the weather) + last 8 turns + user text
  prompt ← chat template(msgs); P ← |prompt tokens|; seed ← crc32(prompt)        // reproducible per input
  out ← sample(temperature ∈ [0.3, 0.5] (default 0.3), top-p 0.9, max_new_tokens ∈ [8,160] (chat sends 60),
               logits processor = RepetitionPenalty(1.15, prompt_ignore_length = P),
               stop on "\n\n", a speaker tag, cancellation or deadline)
  t ← CLEAN(out)  // cut at end tokens, blank line, second speaker tag; drop own "Name:"; trim unfinished sentence
  v ← VALIDATE(t, ...);  return {text: t, accepted: v = ∅, violations: v}
  // busy queue, timeout, not loaded → {fallback: true, reason}; caller shows deterministic text
```

The standard repetition penalty divides (or multiplies, for negative logits) the logit of *every* token already present in the sequence, including the prompt. Here the prompt contains the grounded facts and the draft, so the stock penalty pushes the model away from exactly the numbers it must repeat. `prompt_ignore_length = P` restricts the penalty to tokens generated after the prompt.

## 4. Flowchart and state machine

The validator is a pure function: it keeps no state between calls, and the same text and evidence always give the same verdict. Figure 13.1 is therefore a flowchart of one call, with the decision that the service and SwarmChat take on its result. The state that does exist belongs to the generation service around it, whose bounded queue and deadlines decide whether a sentence reaches the validator at all (Figure 13.2).

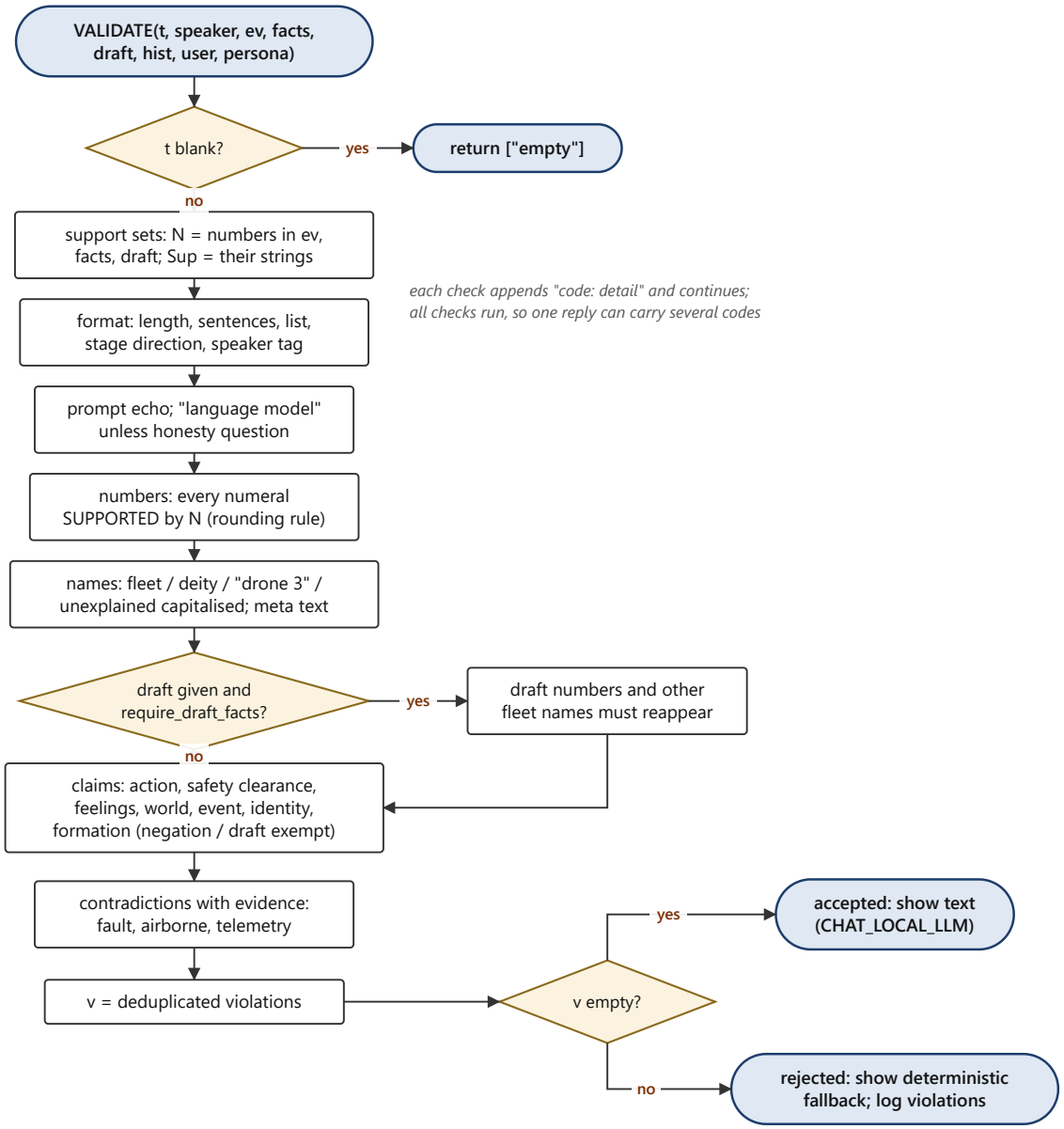



**Figure 13.1** Flowchart of `validate` (rule groups in code order) and of the accept/reject decision made by the service and its caller. Only a blank text short-circuits.

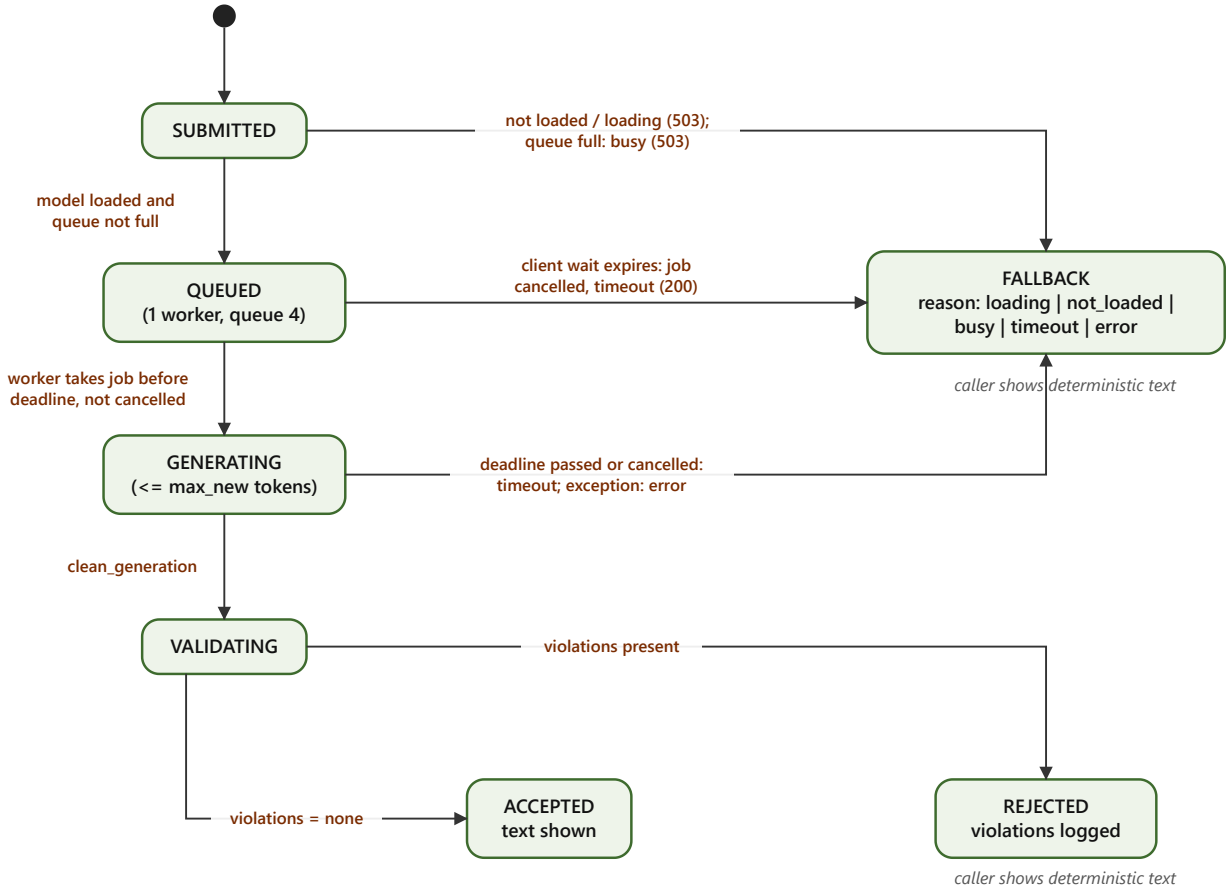



**Figure 13.2** Life cycle of one wording request in `chat_llm_service.py` (`submit`, `_run`, `_handle`). The validator itself is stateless; the state belongs to the bounded request queue. Every terminal state is logged append-only.

> **Data flow.** SwarmChat (Algorithm 12) builds *evidence* (telemetry fields, recorder history, fixed architecture statements) and, for history and follow-up replies, a deterministic *draft*. The service turns both into the prompt, samples at most 60 new tokens, cleans the text and calls `validate` with the same evidence and draft. User text and chat history are passed too, but they can only make *names* acceptable, never numbers or shapes.

## 5. Parameters

| Parameter | Value |
|---|---|
| Length / sentence limits | 400 chars or \|draft\|+80; 3 sentences or the draft's count |
| Airborne threshold (evidence) | alt/z ≥ 0.30 m, else mode families |
| Negation window | same clause, last 6 words (≤ 60 characters back) |
| Integer rounding allowed | only for \|value\| ≥ 10 (87.3 → "87"; never 1.5 → "2") |
| Repetition penalty | 1.15 on generated tokens only |
| Sampling | temperature clamp 0.3–0.5, top-p 0.9, crc32(prompt) seed |
| Service bounds | 1 worker, queue 4, 12 s default timeout, body ≤ 64 KiB, loopback only, refuses ports 8100/8765/8766 |

## 6. Complexity

Validation is O(|t| + |support|) per regular expression with a fixed set of patterns, plus O(|numerals(t)|·|N|) for numbers and O(|fleet|·|support|) for names; sub-millisecond to milliseconds for chat-length texts. Generation dominates (≤ 60 new tokens on CPU).

## 7. Properties

### Guaranteed

- An accepted sentence contains no numeral that is not equal to, or a permitted rounding of, an evidence/draft/allowed number; no fleet name unknown to the conversation; no shape word unsupported by evidence in a formation context; and none of the listed authority, feelings, identity, event, world or contradiction patterns (outside negation and verbatim-draft exemptions).
- User text and history never make a number or a shape "supported".
- Deterministic and side-effect free; the same text and evidence always give the same verdict.
- Rejected or unavailable generations are replaced by deterministic text; the decision and reasons are logged append-only.

### Not guaranteed

- Pattern-based: semantic falsehoods expressed without a flagged numeral, name, shape or keyword (e.g. a wrong comparison "Ra is higher than Thoth" when both values are in evidence) are not detected.
- Spelled-out numbers are not checked. Negation detection is a word-window heuristic and can be fooled.
- The draft exemption trusts the deterministic draft entirely.
- The repetition-penalty change affects style and fact retention statistically; it guarantees nothing per sentence.

## 8. Testing and measured results

- `acceptance/tests/test_chat_grounding.py` (41 tests: accepted good texts, real observed failures, one test per violation class, helpers, "no torch import"); runner `acceptance/run_chat_grounding_tests.sh`. Run results are **not recorded** in `acceptance/results`.
- Repetition-penalty ablation, `acceptance/llm_bench/bench_smollm.py` (35 prompts, hand-scored 0–2 on naturalness, persona, facts, boundaries, context; seed-1 reply; files `rubric_*.json`): SmolLM2-360M-Instruct mean total 4.03/10 with the stock penalty vs 4.77/10 with the generated-only penalty; the facts dimension rose from 0.17 to 0.46 (of 2). For SmolLM2-1.7B-Instruct: 6.69 vs 6.77; facts 0.83 vs 0.89. Single seed and a single human scorer: indicative only.
- Service logs (`logs/chat_llm/`, 11 files, 2026-11-11 05:47–15:37 UTC): 24 completed generations, 13 accepted (54 %), 15 fallbacks (busy/timeout/not loaded); most frequent rejection codes number_not_in_evidence (6) and missing_draft_fact (5). Models in these logs were SmolLM2-135M-Instruct (17) and 1.7B-Instruct (7), not the current 360M default.

## 9. Relation to other algorithms

Called for every wording request of Algorithm 12. The safety-clearance rule was added from benchmark case S7 (`scripts/patch_chat_bench.py`). Evidence comes from the telemetry structures of Algorithm 17.

## 10. Comparison with alternatives and related work

Detecting unsupported content in generated text is an active research area, usually called hallucination detection or faithfulness evaluation [1]. The validator belongs to the simplest family: lexical checks of the output against a known source. The table places it next to established alternatives. No alternative was run on the SwarmChat logs; the comparison is qualitative and based on the published descriptions.

| Approach | How it works | Where it is stronger than this validator | Where this validator differs |
|---|---|---|---|
| Sampling-based self-consistency (SelfCheckGPT) [2] | Samples several answers from the same model and scores each sentence by its consistency with the others (BERTScore, QA, n-gram, NLI or prompt variants); no external source needed. | Works without any evidence record; catches semantic errors that contain no flagged token; graded scores rather than a binary verdict. | Here the source of truth is known exactly (the evidence record), so support can be checked directly and deterministically. Self-consistency needs several generations per reply, which a single-worker CPU service with a 14 s budget cannot afford, and a consistently wrong model passes it. |
| Atomic-fact verification against a knowledge source (FActScore) [3] | Splits the text into atomic facts and checks each one against a source, by humans or by a retriever plus a strong LM. | Semantic checking of arbitrary claims, including comparisons and relations; a precision score per text. | No decomposition model and no second LM: the validator checks tokens (numerals, names, shape words, keyword patterns). It cannot detect a wrong statement that uses only supported tokens (e.g. swapping which aircraft is higher). |
| Table-grounded overlap metrics (PARENT) [4] | Data-to-text evaluation that aligns n-grams of the output with the source table as well as the reference, giving precision and recall. | Continuous score, recall as well as precision, validated against human judgements on WikiBio. | Used here as a run-time gate, not an evaluation metric. Only precision-like support is enforced (plus `missing_draft_fact` as a narrow recall check); there is no score, only accept/reject. |
| Programmable output rails (NeMo Guardrails) [5] | Output rails run checks on a reply before it is returned; the toolkit describes, among others, fact-checking and hallucination rails that use an LLM. | Configurable, model-independent framework with several ready-made rails. | Same placement in the pipeline, but the checks are fixed Python patterns with no LLM in the loop, so the gate is deterministic, fast and testable, and can be re-run independently by the observer process. |

The repetition-penalty change is related to the penalised sampling introduced with CTRL [6], which discounts every token already present in the context; restricting it to generated tokens (`prompt_ignore_length`) keeps the penalty from discouraging the copying of evidence numbers.

> **Where this design is weaker.** It is purely lexical. It misses false comparisons and relations built from supported tokens, spelled-out numbers, paraphrased incidents not in its keyword lists, and anything the deterministic draft gets wrong (the draft is trusted). Negation handling is a 6-word window. The rule set was tuned on a small number of observed failures and a 35-prompt single-scorer benchmark; unlike the methods above it has no published evaluation against human labels, and its false-rejection rate (11 of the 24 completed generations in the logged sessions, 46 %, were rejected) is high.

**Honesty notes.** An accepted sentence is not shown to be true; it is shown to contain no unsupported numeral, name, shape word or flagged pattern. The validator does not make the wording model reason, and generated chat cannot actuate anything (Algorithm 12).

## References


1. Z. Ji, N. Lee, R. Frieske, et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys* 55(12), Article 248, 2023. https://doi.org/10.1145/3571730
2. P. Manakul, A. Liusie, M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," *Proc. EMNLP 2023*, pp. 9004–9017, 2023. https://aclanthology.org/2023.emnlp-main.557
3. S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," *Proc. EMNLP 2023*, pp. 12076–12100, 2023. https://aclanthology.org/2023.emnlp-main.741
4. B. Dhingra, M. Faruqui, A. Parikh, M.-W. Chang, D. Das, W. W. Cohen, "Handling Divergent Reference Texts when Evaluating Table-to-Text Generation," *Proc. ACL 2019*, pp. 4884–4895, 2019. https://aclanthology.org/P19-1483

5. T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, J. Cohen, “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails,” *Proc. EMNLP 2023: System Demonstrations*, pp. 431–445, 2023. https://aclanthology.org/2023.emnlp-demo.40
6. N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, R. Socher, “CTRL: A Conditional Transformer Language Model for Controllable Generation,” arXiv:1909.05858, 2019. https://arxiv.org/abs/1909.05858

---

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.